Milestone Three

Algorithms and Data Structure Artifact

Matthew Schaub

SNHU

CS 499

05/31/2026

The artifact for algorithms and data structure is my OpenGL 3D scene project originally created in my CS 330 Computational Graphics and Visualization course. This project was created using C++ and OpenGL. The program renders a 3D scene using shapes, textures, lighting, camera controls, and projection settings. In this project, we see the use of arrays, structured logic, object rendering, and OpenGL resource management.

I selected this artifact for my ePortfolio because it is a working program that shows more than just a visual scene. It shows how code can be used to organize textures, manage objects, and control how a scene is displayed. This makes it a good artifact for algorithms and data structure because the program stores texture information in an array, uses logic to control rendering behavior, and requires safe handling of resources.

The artifact was improved in several ways. One improvement was correcting the orthographic projection logic. In the original version, the function always returned false instead of checking the actual projection state. I corrected this so the program now returns the stored projection value. This made the logic more accurate and easier to understand.

Another improvement was fixing the texture cleanup process. The original version used glGenTextures in the cleanup function, which creates textures instead of deleting them. I changed this to glDeleteTextures, reset the texture IDs, cleared the texture tags, and reset the loaded texture count. This improved the resource management of the program.

I also added boundary checking to the texture loading process. Since the program uses a fixed texture array, I added a check to prevent more textures from being loaded than the array can safely hold. I also improved error handling for unsupported image formats so the image

memory is released before the function exits. These changes made the program safer and more reliable.

In Module One, I identified course outcomes that my enhancements would align with. This enhancement makes the strongest progress toward course outcome 3 because it connects directly to algorithms and data structures. The program uses arrays, logic, and structured rendering decisions to solve a graphics problem. Course outcome 4 was also demonstrated through my use of C++, OpenGL, Visual Studio, shaders, and textures to improve a working application. Course outcome 5 was supported through defensive programming improvements such as boundary checking, safer cleanup, and better handling of invalid texture data.

An update on my original plan is that I focused on practical improvements that made the program more stable and easier to review. I did not fully restructure the entire rendering system because I wanted to avoid breaking the existing scene. A future improvement would be to move repeated rendering logic into reusable helper methods, but the current changes were a solid step forward for this milestone.

This process taught me that a program can work visually but still have problems in the code behind it. When I first created the project, my main goal was to get the scene to render correctly. Reviewing it now helped me look at the project more like a developer reviewing legacy code. I learned the importance of checking logic, managing resources correctly, validating boundaries, and writing code that another developer could understand.

The biggest challenge was making improvements without breaking the existing 3D scene. OpenGL projects have many connected parts, including shaders, meshes, textures, lighting, and camera controls. Overall, these enhancements made the artifact stronger for my ePortfolio

because it now shows growth in algorithms and data structures, software design, and defensive programming.